

F6: Arv, generalisering

Malmö universitet, Institutionen för datavetenskap och medieteknik

Innehåll

- Repetition
- Arv
- Arv i kod
- Avsnitt 3
- Sammanfattning



Repetition

Repetition

- Vi har pratat om objekt och klasser
- De flesta exemplen har varit i stil med ”objekt A har objekt B”
 - En person har en cykel
 - Ett stjärnsystem har planeter
- Vi ska nu kolla på när ”objekt A är objekt B”
 - En cykel är ett fordon
 - En hund är ett husdjur
 - En man är en person

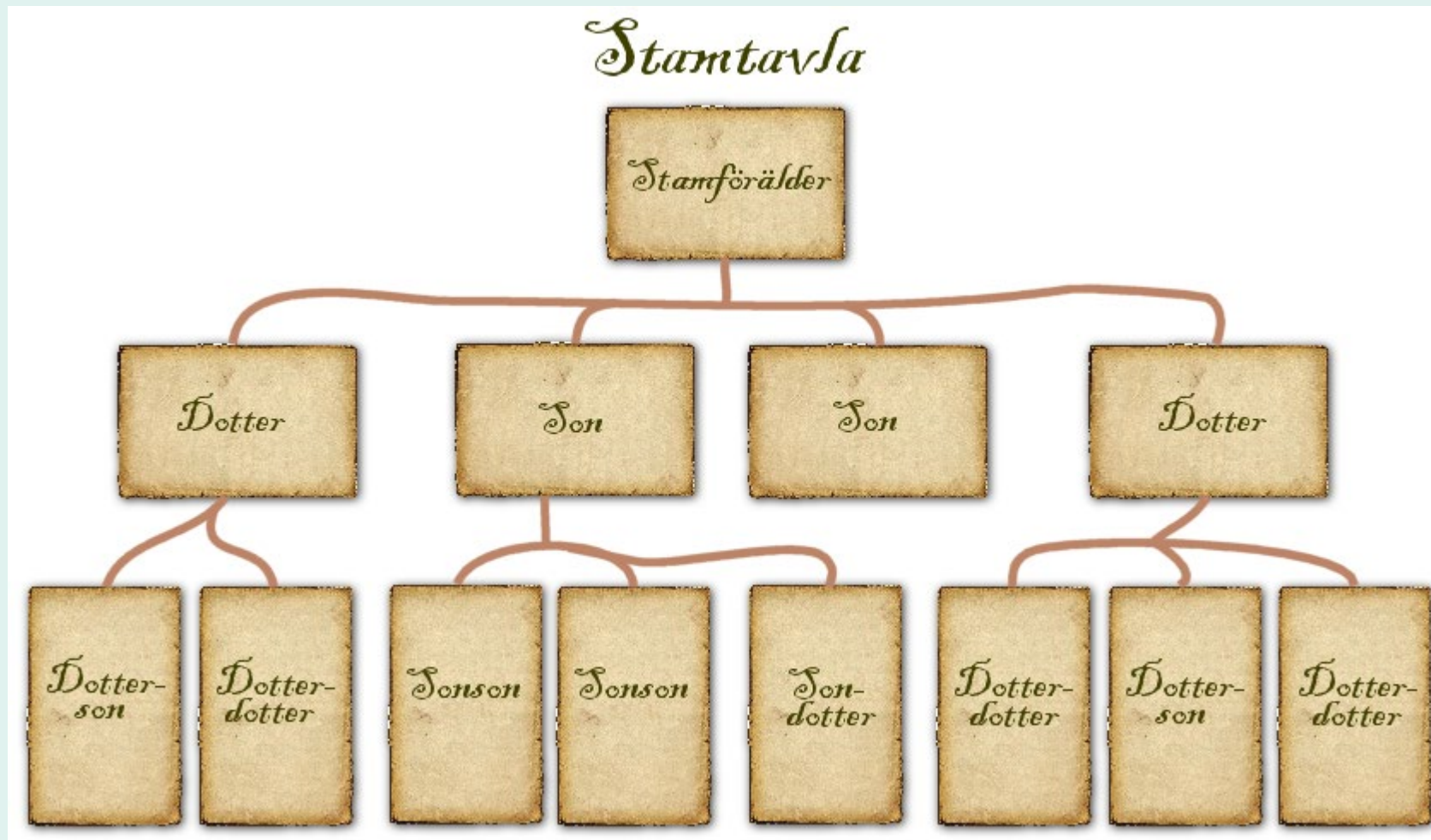


Arv

Saker Calle inte får ta upp

- Polymorfism
 - Det som gör arv riktigt användbart
 - Förenklat: objekt som ärver från samma klass kan ligga i samma array
- Abstrakta klasser
 - En klass som inte kan skapa objekt
 - Måste bli ärvd
- Interfaces
 - En variant av arv
- Allt detta kommer nästa föreläsning

Arv irl



Arv i programmering

- Vi kan ha situationer där flera objekt är snarlika
- Flera gemensamma attribut
- Flera gemensamma operationer

Minecraft

- I spelet *Minecraft* finns det flera sorters Block som bygger upp världen
 - Rock, Dirt, Iron Ore, Copper Ore, Coal o.s.v.
- Dessa har alla likheter och behandlas väldigt snarligt i spelet
- Alla har:
 - Position
 - Hit points
 - Texturer
- Samma gäller verktyg
 - Svärd, Pick axe, Axe, Hoe

Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Vad är detta:



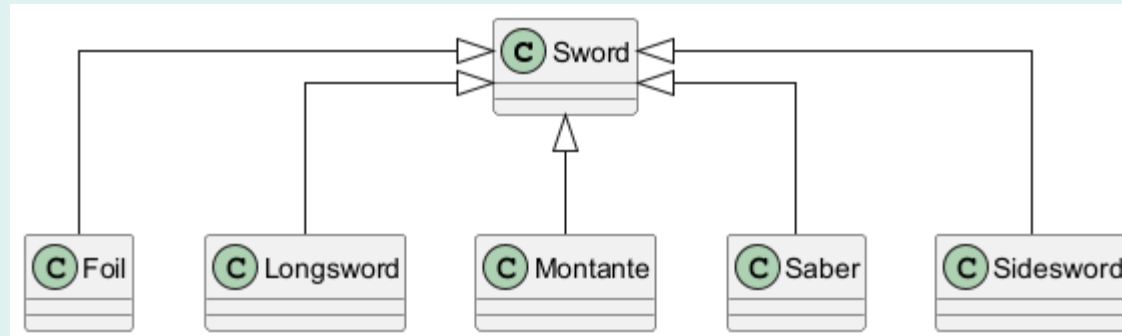
Mer vardagligt exempel

- Vad är detta:



Mer vardagligt exempel

- Alla bilderna föreställde svärd
- Alla svärden var sina egna objekt
- Alla svärden var inte av samma typ

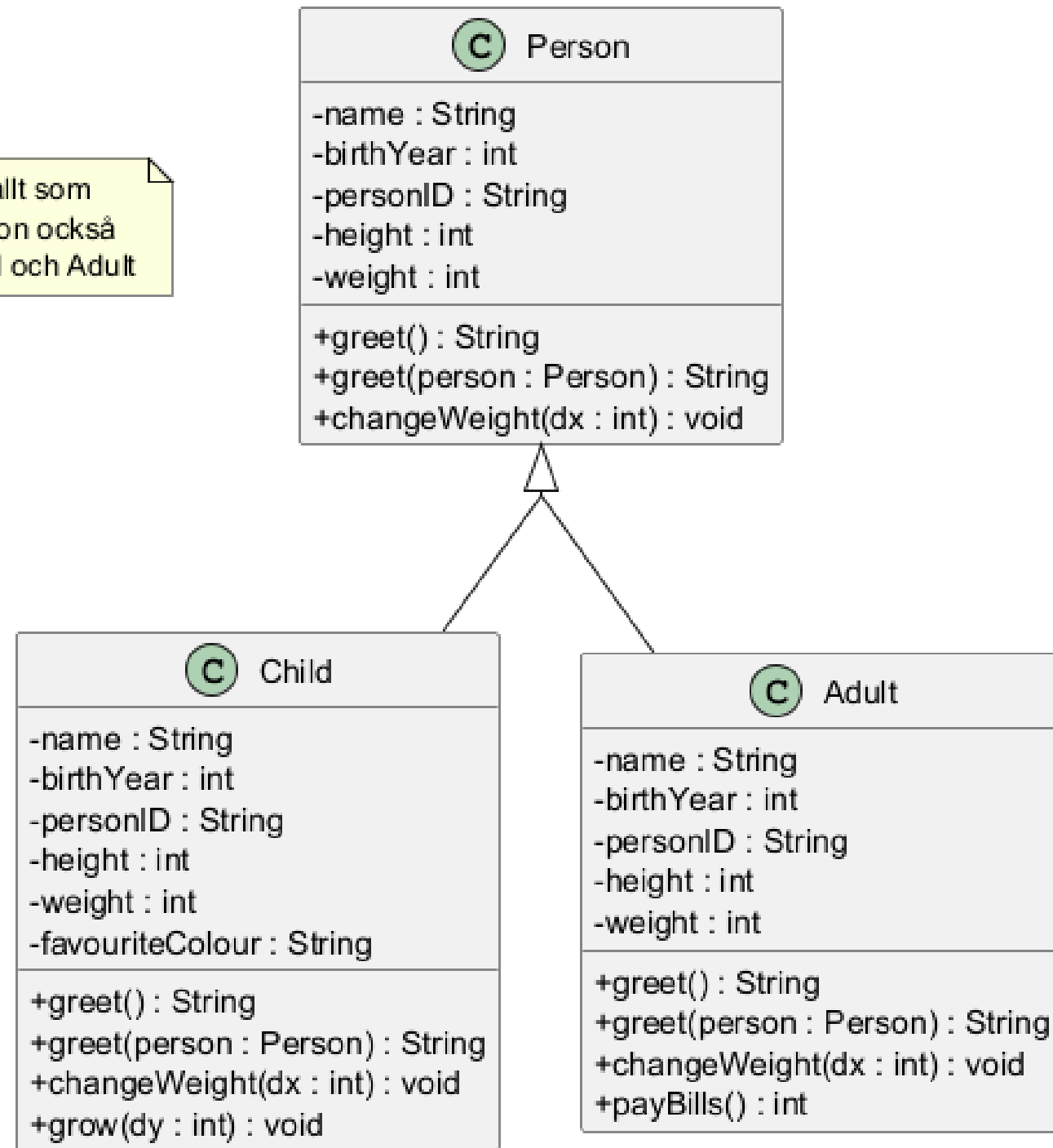


- *Bonusfakta: Människor är väldigt bra på kategorisering*

Klassdiagram

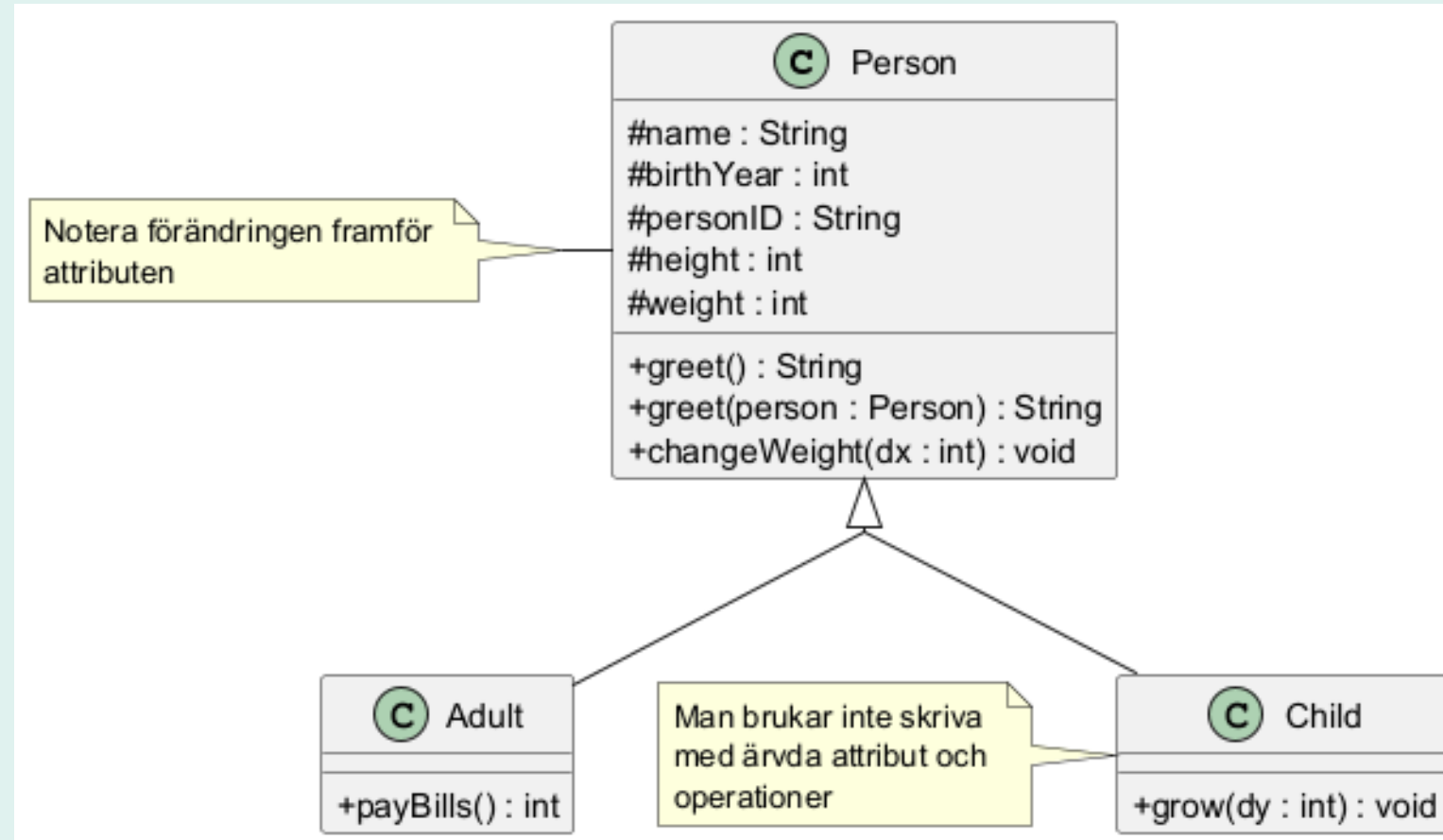
- Arv markeras med en tom pil

Notera att allt som finns i Person också finns i Child och Adult



Klassdiagram

- # markerar att ett attribut eller en operation är *protected*
- Kom ihåg att *private* **bara** var tillgängligt i den egna klassen
- Ett *protected* attribut eller operation är tillgänglig i subklassen

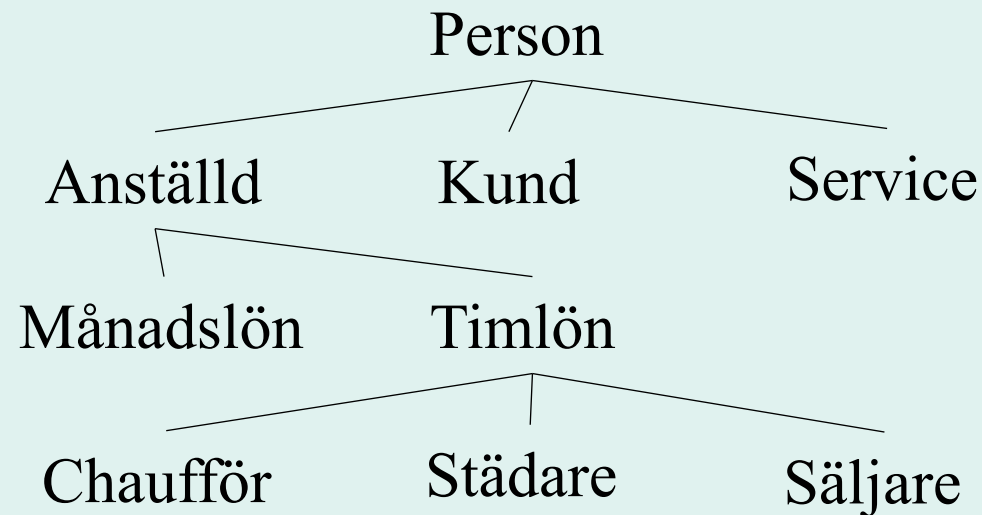


Super- och subklass

- Klassen som ärvs ifrån kallas för en *superklass*
- Klassen som ärver kallas för en *subklass*
- I klassdiagram är *superklasser* oftast ovanför *subklasser*

Ett till exempel (ett företag)

- En person kan vara en anställd, kund eller utföra en tjänst åt företaget
- En anställd kan vara månadsavlönad eller timavlönad
- En timavlönad (i det här företaget) kan vara chaufför, städare eller säljare



Mer generella
(superklasser)



Mer specialiserade
(subklasser)



Arv i kod

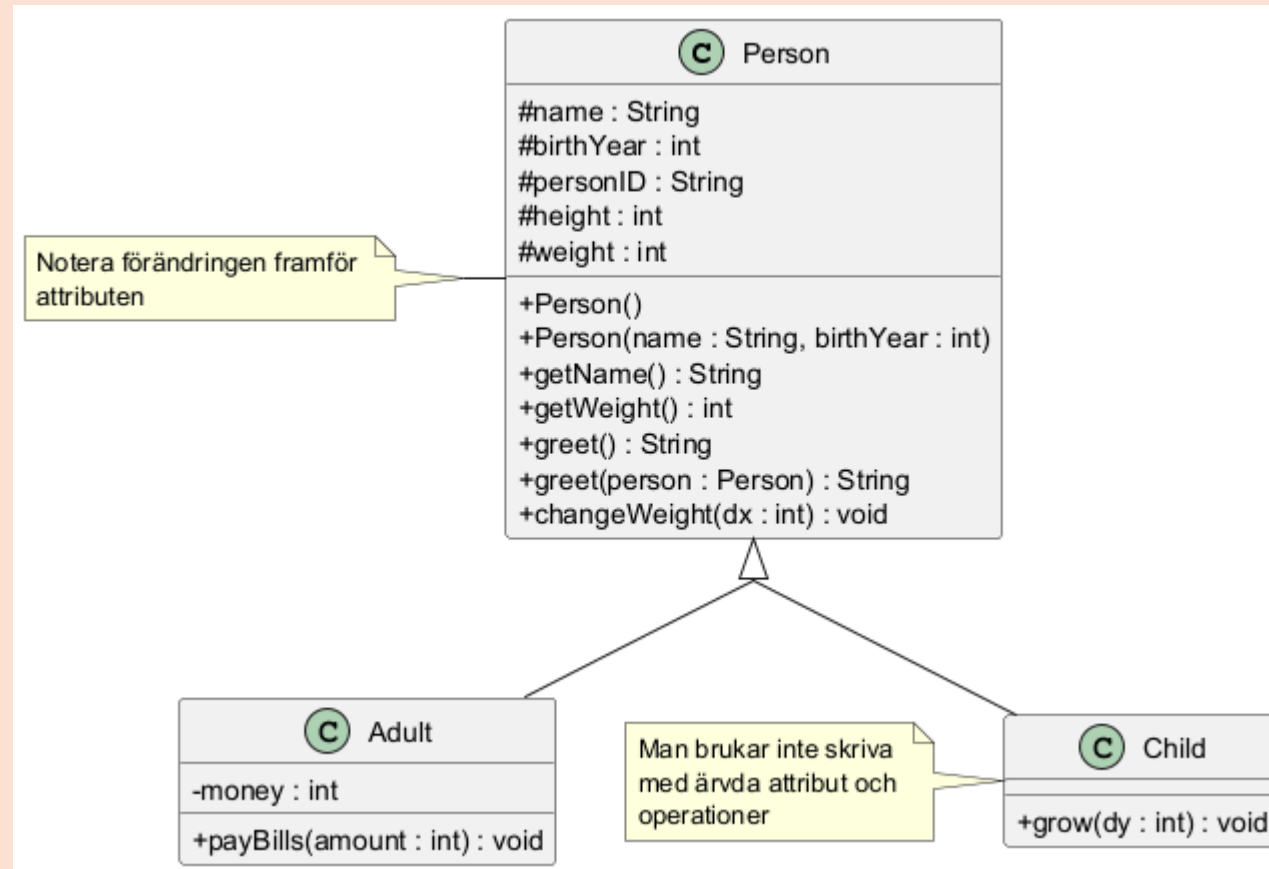
Arv vs. generalisering

- Konceptet med superklasser och subklasser kallas för *generalisering*
- Implementeringen i kod kallas *arv*
- Begreppen används ofta slarvigt och blandas lätt ihop
- I Java säger man dessutom att en klass *extends* en annan klass när den ärver från klassen

Att skapa subklasser

- Vi kan ärva från alla klasser
- Vi behöver alltså inte markera det på något sätt
- I Java skriver man att en subklass *extends* en superklass
- TODO: Visa exempel på en klass som extendar en annan

Klasserna Person, Adult och Child



Person

- Den här klassen gör inget spännande utöver:

```
public String greet(Person person){  
    return "Hello " + person.getName() + ", my name is "  
+ name;  
}
```

- All koden läggs upp på Canvas

Adult

```
public class Adult extends Person{  
  
    private int money;  
  
    public Adult(String name, int birthYear){  
        super(name, birthYear);  
    }  
  
    public void payBills(int amount){  
        money -= amount;  
    }  
}
```

Child, del 1

```
public class Child extends Person{

    public Child(String name, int birthYear){
        this.name = name;
        this.birthYear = birthYear;
    }

    public void grow(int dy){
        this.height += dy;
    }
}
```

Child, del 2

```
    public String greet(){
        return "Hello, my name is " + name + ", and I am
" + height + " cm tall!";
    }
    public String greet(Person person){
        if (person.getName() == name){
            return "WE SHARE THE SAME NAME! WOW WE ARE
BESTIES FOR EVER!";
        }
        return super.greet(person);
    }
}
```

Att kalla på super-klassen

- I koden ovan kallade vi på super-klassen på två ställen
 - Konstruktorn i **Adult**
 - Den ena *greet*-metoden i **Child**
- Vi anropar super-klassens konstruktor på samma vis som vi *delegerar* en konstruktor vanligtvis
- I övrigt kan den användas som *this*.





Metoder och arv

Tre situationer

1. En sub-klass har en ingen egen version av metoden
2. En sub-klass har en egen version av metoden (*override*)
3. En sub-klass kallar på super-klassens metod

Fall 1, sub-klassen saknar metoden

```
Adult adult1 = new Adult("Mufasa", 2024);  
  
String n = adult1.getName();
```

- I det här fallet anropas metoden *getName()* i super-klassen **Person**.

Fall 2, sub-klassen implementerar

```
Child child1 = new Child("Kiara", 1998);
```

```
String g = child1.greet();
```

- I det här fallet anropas metoden *greet()* i sub-klassen **Child**.
- Detta kallas för en *override* och kan markeras på följande vis i Java:

```
@Override  
public String greet(){ ... }
```


Fall 3, sub-klassen anropar super

```
Child child1 = new Child("Kiara", 1998);
```

```
String h = child1.greet(person1);
```

- I det här fallet anropas metoden *greet(Person)* i sub-klassen **Child**.

```
public String greet(Person person){  
    if (person.getName() == name){  
        return "WE SHARE THE SAME NAME! WOW WE ARE  
BESTIES FOR EVER!";  
    }  
    return super.greet(person);  
}
```



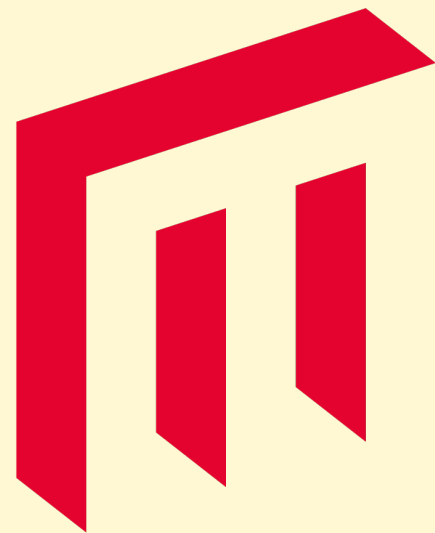
Sammanfattning

Viktiga koncept

- Begreppen *arv*, *generalisering*, *extends*
- Begreppen *super-* och *subklasser*
- Representation i klassdiagram
- Begreppet *protected*
- Enklare implementering
 - metoder
 - *override*

Rekommenderad läsning

- Deitel & Deitel, kapitel 9, s. 383–416
- Michel Ende, *Momo eller kampen om tiden*



**MALMÖ
UNIVERSITET**

